Bachelor's Thesis Computing Science

Radboud University Nijmegen

Thesis Title

Subtitle if you like

Author:
Kunal Narwani
s1102120

First supervisor/assessor:
Title Twan van Laarhoven

Second assessor:
Title Gabriel Bucur

November 26, 2025

Abstract

Brief outline of research questions, results. (The preferred size of an abstract is one paragraph or one page of text.)

Contents

Chapter 1

Introduction

Problem Statement Deep Convolution Neural Networks (CNNs) have set the state of the art in the image-to-image translation task, most notably in concepts like Super-Resolution (SR) and Image Segmentation. Much of this progress was driven by the introduction of the U-Net architecture, characterized by its contracting path to capture context and its symmetric expanding path with skip connections for precise localization. This structure has proven effective across both domains, including biomedical image segmentation[12] and image super-resolution (SISR)[10].

However, when applied across a broad spectrum of inputs, the current implementation of the U-Net faces some architectural limitations.

- Fixed Capacity and Variable task difficulty: U-Net architectures are designed with a fixed number of encoder-decoder levels (fixed depth). For deep networks in general, the rule is that greater depth generally leads to better performance but significantly higher computational cost and it leads to slower convergence because of the large number of parameters.
- Sub-Optimal Scalling, Loss of Spatial detail and Stability: The vanilla U-Net architecture consists of a pooling and downsampling path, which present difficulties when handling fine details and adpating to different scales. Many deep networks (DN) designed for image task use max-pooling layers which can reduce localization accuracy[12]. When features are repeatedly downsampled in the encoder, low-level details fade. As a result, the reconstructed high-resolution image is often missing fine textures.
- The Need for Differentiable Scaling and Improved Operators: The need for improved scaling mechanisms stems directly from the limitations of standard operations in CNNs, such as max pooling, which is used in the contracting path of the original U-Net. Conventional pooling operations, including both max pooling and mean pooling, lose some important feature information when processing feature maps. Specifically:
  - Max Pooling selects only the maximum value, which can be sensitive to noise.
  - Mean Pooling smooths the image and leads to the loss of high frequency information.

  Moreover, the traditional U-Net relies on fixed pooling, upsampling operations (like transposed convolutions), or standard interpolation methods that are discrete and non-differentiable with respect to scale. This dependence on fixed-stride operations limits the network's flexibility and spatial precision, particularly when handling non-integer scale factors. Therefore, there is a requirement for differentiable resizing layers (such as ResizeByScale and ResizeToMatch) that perform continuous spatial scaling using methods like bilinear interpolation, enabling the network to adapt its depth and spatial scaling dynamically during training while maintaining smooth gradient flow.

These limitations highlight the requirement for architectural innovations that allow the U-Net framework to dynamically adapt its internal structure—including its depth and scaling operators—to the specific input scale and task demands, thereby improving feature retention and reconstruction quality

Primary research question How does variation in scale factor impact U-Net performance?

Sub research question How does the proposed adaptive-depth U-Net architecture, which adjusts its number of encoder-decoder levels based on the scale factor (s), compare against a fixed-depth U-Net baseline in the field of super-resolution and segmentation?

- motivate why this problem must be solved

  Solving this problem is important because current U-Net–based systems implicitly assume that a single fixed architectural depth is equally suitable for all input scales. However, it is not evident that this assumption holds in scenarios where the amount of available spatial information varies strongly with the scale factor. Inputs with limited detail—such as smooth backgrounds or heavily downscaled images—may gain little from deep encoder–decoder hierarchies, which risk collapsing feature maps and discarding fine information. In contrast, inputs containing richer structure and high-frequency content may benefit from deeper networks that can extract more complex features. By examining how performance changes across different scale factors, we can determine whether a fixed-depth U-Net is sufficient or whether different input regimes demand different architectural capacities. Addressing this question helps clarify whether adaptive architectures are necessary for optimal reconstruction and segmentation across varying levels of spatial detail.

- demonstrates that a (new) solution is needed

  Traditional U-Net architectures rely on fixed-depth encoder–decoder pathways and discrete downsampling and upsampling operators such as max pooling and transposed convolutions. These design choices work well when training and inference occur at a single, fixed resolution, but they become limiting when the scale factor varies. Fixed depth forces the same capacity on inputs with vastly different spatial complexities, while fixed-stride operations are not differentiable with respect to scale and often misalign feature maps in the skip connections. As the scale factor changes, these limitations can lead to either an unnecessary loss of spatial detail or an inefficient overuse of computational resources. Because existing U-Net variants do not adapt their internal structure to the input scale, they cannot explicitly address these issues. This gap motivates the development of a U-Net architecture whose depth and scaling operators adjust dynamically to the characteristics of the input.

- explain the intuition behind your solution

  The key intuition behind my solution lies in resolving the mismatch between the fixed capacity of max pooling of traditional deep learning architectures and the variable complexity in the input. The primary intuition is that the ideal network depth should not be fixed, but should adapt most based on the input scale factor (s). The vanilla U-Net architecture consists of a fixed depth structure, which is suboptimal when dealing with a wide range of inputs for super-resolution.

- motivate why / how your solution solves the problem (this is technical)

  My thesis solves this problem by introducing a dynamic structure influenced by the input scale s. The motivation for the solution is linked to two technical phases of investigating the limitations of fixed depth (the baseline) and implementing adaptive depth using continuous, differentiable scaling layers (the core technical contribution).

  Motivation of the technical phase 1: The core technical problem stems from the vanilla U-Net using 2×2 max pooling with a stride of 2. My thesis first investigates the consequences of this fixed structure by measuring/verifying the influences of scale on the network

with fixed depth. This established the baseline performance ceiling of the non-adaptive architecture.

Motivation of the technical phase 2: The solution technically links the network capacity (depth) to the spatial resolution required, defined by the scale factor.

- explain how it compares with related work

Close the introduction with a paragraph in which the content of the next chapters is briefly mentioned (one sentence per chapter).

Starting a new paragraph is done by inserting an empty line like this.

Primary research question How does the proposed adaptive-depth U-Net architecture, which adjusts its number of encoder–decoder levels based on the scale factor (s), compare against a fixed-depth U-Net baseline in the field of super-resolution and segmentation?

- **Motivate why this problem must be solved.**

  Solving this problem is important because current U-Net–based systems implicitly assume that a single fixed architectural depth is equally suitable for all input scales. It is not clear that this assumption holds when the amount of available spatial information varies strongly with the scale factor. Inputs with limited detail, such as smooth backgrounds or heavily downscaled images, may gain little from deep encoder–decoder hierarchies, which risk collapsing feature maps and discarding fine information. In contrast, inputs containing richer structure and high-frequency content may benefit from deeper networks that can extract more complex features. By examining how performance changes across different scale factors, this thesis investigates whether a fixed-depth U-Net is sufficient or whether different types of inputs require different architectural capacities. Addressing this question clarifies whether adaptive architectures are necessary for optimal reconstruction and segmentation across varying levels of spatial detail.

- **Demonstrate that a (new) solution is needed.**

  Traditional U-Net architectures rely on fixed-depth encoder–decoder pathways and discrete downsampling and upsampling operators such as max pooling and transposed convolutions. These design choices work well when training and inference occur at a single, fixed resolution, but they become limiting when the scale factor varies. A fixed depth forces the same capacity on inputs with vastly different spatial complexities.

  Operations like max pooling with a fixed stride select the maximum value from a local window; therefore, they do not provide a smooth mathematical relationship (a derivative) describing how the output would change if the scale factor (s) were varied continuously. This makes them non-differentiable with respect to scale and can misalign feature maps in the skip connections. Thus, a new solution is needed in which interpolation-based methods—which are differentiable—are used to provide a smooth mathematical relationship, allowing the neural network to adjust and optimize gradients.

  As the scale factor changes, these limitations can lead either to unnecessary loss of spatial detail or to inefficient overuse of computational resources. Because existing U-Net variants do not adapt their internal structure to the input scale, they cannot explicitly address these issues. This gap motivates the development of a U-Net architecture whose depth and scaling operators adjust dynamically to the characteristics of the input.

- **Explain the intuition behind your solution.**

  The key intuition is to resolve the mismatch between the fixed capacity of conventional U-Net architectures and the variable complexity of inputs at different scale factors. Depth

is treated as a resource that should be allocated according to how much spatial information the input still contains. For very small-scale factors, a deep encoder–decoder quickly reduces feature maps to tiny spatial sizes, increasing the risk of losing fine-grained structure; in such cases, a shallower network is both more stable and more efficient. For larger-scale factors, however, the input contains richer high-frequency content, and deeper networks can exploit this by learning more complex hierarchical features, but this increases the model size and computation required to train. The proposed adaptive-depth U-Net therefore treats the scale factor s as a signal that modulates how many encoder–decoder levels are used, rather than keeping this number fixed.

- **Motivate why / how your solution solves the problem (technical).**

  This thesis addresses the limitations of fixed-depth U-Nets in two technical phases. In the first phase, a vanilla U-Net with 2×2 max pooling and a fixed number of encoder–decoder levels is used as a baseline. By systematically varying the scale factor s while keeping the depth fixed, the experiments quantify how performance degrades or saturates when the architecture cannot adapt its capacity to the input resolution. This establishes the performance ceiling of a non-adaptive architecture and reveals where fixed depth is either wasteful or insufficient.

  In the second phase, the network depth is explicitly linked to the scale factor s through an adaptive-depth design, implemented using continuous, differentiable resizing layers (such as `ResizeByScale` and `ResizeToMatch`) instead of max pooling and transposed convolutions. These interpolation-based layers preserve spatial alignment and allow feature maps to be scaled smoothly for arbitrary s, while the depth schedule assigns fewer levels to heavily downscaled inputs and more levels to high-detail inputs. In combination, scale-aware depth and differentiable scaling operators directly address the earlier identified problems: they reduce unnecessary feature-map collapse at small scales, allocate additional capacity where it is beneficial, and maintain stable gradient flow across varying resolutions.

- **Explain how it compares with related work.**

  The proposed architecture builds on three main lines of prior work: the original U-Net for biomedical segmentation, U-Net variants adapted for super-resolution, and architectures that refine pooling and skip connections. Classical U-Net papers focus on fixed-depth designs at a single resolution, and super-resolution variants typically modify loss functions or introduce residual heads, but still keep depth and scaling operators fixed across scales. Other work improves feature rescaling or skip connections (for example via interpolation-based pooling or adaptive skip pathways), yet does not explicitly couple network depth to the input scale factor. In contrast, this thesis combines a residual U-Net backbone with differentiable interpolation-based scaling layers and a scale-dependent depth schedule, and evaluates the resulting architecture jointly on super-resolution and segmentation. This positioning allows a direct comparison with fixed-depth U-Nets and existing U-Net-style super-resolution models, while highlighting the specific contribution of scale-conditioned depth as an architectural degree of freedom.

Chapter 2

Preliminaries

This chapter introduces the fundamental concepts and definitions required to understand the proposed methodology. It defines the problems of image super-resolution and image segmentation, summarizes essential deep learning concepts, and describes the objective functions (loss function) and evaluation metrics used throughout this thesis.

2.1 Image Super-Resolution

Image super-resolution (SR) aims to reconstruct a high-resolution (HR) image from a given low-resolution (LR) input. Traditional interpolation methods such as bicubic upsampling often yield blurry and over-smoothed results, as they rely on fixed mathematical assumptions rather than learned patterns. Deep learning models learn data-driven mappings that reconstruct missing high-frequency details from low-resolution inputs. The SR task is inherently ill-posed, since multiple HR images can correspond to the same LR observation. Therefore, the goal of modern SR models is to learn a mapping function

$$Y = f_\theta(X),$$

where X is the LR input, Y the ground-truth HR image, and f_θ a neural network with learnable parameters θ that minimizes the difference between predicted and target images.

2.2 Image Segmentation

Image segmentation assigns a class label to every pixel in an image, dividing it into semantically meaningful regions. In biomedical imaging, segmentation is used to delineate anatomical or pathological structures such as lesions or organs. In this thesis, segmentation serves primarily as an auxiliary task to evaluate the cross-domain generalization of the proposed adaptive-depth U-Net. Because both super-resolution and segmentation employ encoder–decoder architectures with skip connections, a single unified framework can effectively support both [12, 9].

2.2.1 Skin Lesions in Dermoscopic Imaging

Skin lesions are localized abnormalities of the skin that can indicate benign or malignant conditions. In dermoscopic images, the lesions exhibit large variations in color, texture, and sharpness of the boundaries, often visually overlapping with the surrounding tissue. Accurate segmentation of the lesion area provides a useful benchmark for model robustness and spatial precision, but is not the primary focus of this work. The ISIC-2017 dataset offers standardized dermoscopic images and corresponding lesion masks for objective evaluation of segmentation performance [11].

2.3 Core Deep Learning Components

This section introduces the core building blocks used in the U-Net architecture.

Figure 2.1: Qualitative super-resolution example. From left to right: ground-truth high-resolution (HR) image, low-resolution (LR) image upsampled by interpolation, model prediction, absolute difference between HR and prediction, and edge-difference (Sobel) map. The bottom row shows zoomed-in crops highlighting the differences between HR, LR-upsampled, and the predicted image.

2.3.1 Convolutional Neural Networks (CNNs)

- **Convolutional Layer:** The core operation of CNNs applies a set of learnable filters (kernels) across the input image to produce feature maps. This enables the network to capture hierarchical spatial features such as edges, textures, and shapes.
- **Activation Functions:** Nonlinear activation functions allow networks to model complex relationships. The most common choice, and the one used in this thesis, is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$.
- **Downsampling:** Pooling or strided convolutions are conventionally used to reduce spatial dimensions, increase the receptive field and enable extraction of contextual information of higher level.
- **Upsampling:** Transposed convolutions or interpolation layers increase spatial resolution to reconstruct fine details of the image during the decoding stage.

2.3.2 The U-Net Architecture

The U-Net architecture, introduced by Ronneberger et al. (2015), was originally developed for biomedical image segmentation [12]. It follows a symmetric encoder–decoder structure:

- **Encoder (Contracting Path):** A stack of convolutional and pooling layers that progressively capture global context.
- **Decoder (Expanding Path):** A series of upsampling and convolutional layers that reconstruct spatial resolution and localize details.
- **Skip Connections:** Feature maps from each encoder level are concatenated with their corresponding decoder level. These skip connections preserve high-frequency information and help the network recover spatial precision lost during downsampling.

This architecture is effective for both super-resolution and segmentation, as it combines semantic context with pixel-level detail reconstruction [12].

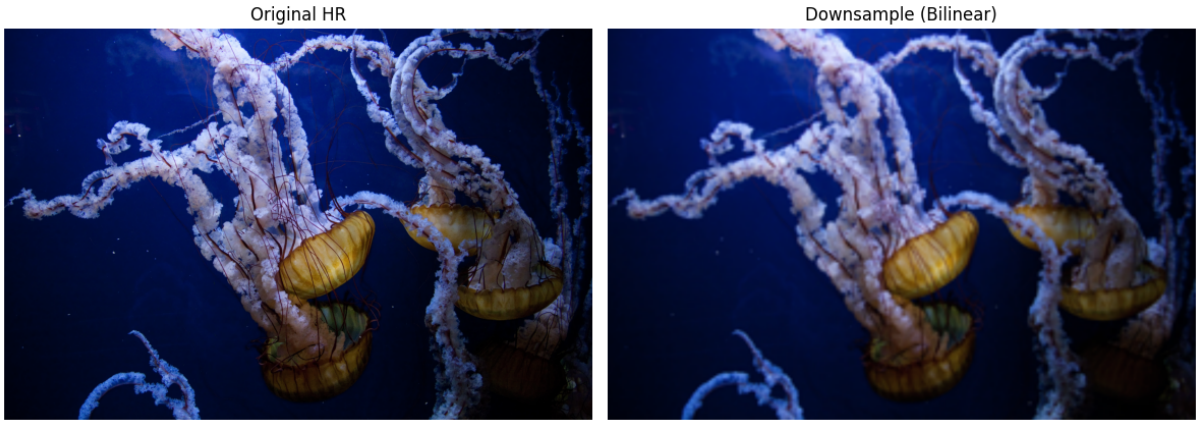

Figure 2.2: Comparison between the original high-resolution (HR) image and its bilinearly downsampled counterpart. Bilinear interpolation produces a smooth but visibly blurred low-resolution image, illustrating the information loss that the super-resolution network aims to reconstruct.

2.3.3 Residual Learning

Residual learning, introduced in ResNet [6], simplifies the training of deep networks by letting layers learn residual corrections instead of full transformations. Rather than directly predicting an output $H(x)$, the model learns a residual function $F(x) = H(x) - x$, and reconstructs the final output as $H(x) = F(x) + x$. This approach improves gradient flow and makes optimization more stable.

In super-resolution, the same principle is applied by predicting only the missing high-frequency details instead of the full high-resolution image. The residual head in this thesis implements this through the `ClipAdd` layer:

$$\hat{Y} = \text{ClipAdd}(X_{LR}, \text{Residual}(X_{LR})),$$

which adds the predicted residual to the low-resolution input and clips the output to ensure all pixel values remain within the valid intensity range $[0, 1]$.

2.3.4 Interpolation-Based Scaling Layers

Traditional U-Net architectures rely on fixed pooling and upsampling operations that are discrete and non-differentiable with respect to scale. Inspired by interpolation-based pooling concepts proposed by Wang [13], this thesis uses differentiable resizing layers that allow smooth and flexible scaling of feature maps.

ResizeByScale. The `ResizeByScale` layer replaces conventional pooling by resizing feature maps according to a continuous scale factor using bilinear interpolation with optional anti-aliasing. This approach preserves spatial alignment and enables flexible downsampling without strided convolutions or max-pooling.

ResizeToMatch. The `ResizeToMatch` layer adjusts one tensor to match the spatial size of another before concatenation in skip connections. It ensures consistent feature alignment between encoder and decoder stages, allowing smooth reconstruction across varying scales.

ClipAdd. In the reconstruction stage, the `ClipAdd` layer adds the predicted residual to the low-resolution input and the clips values to the valid intensity range $[0, 1]$. This operation implements residual learning in a stable and numerically safe form.

Together, these layers replace non-differentiable resizing operations with smooth, learnable interpolation, enabling the network to adapt its depth and spatial scaling continuously during training.

2.4 Loss Functions and Evaluation Metrics

Learning objectives and evaluation criteria are defined through task-specific loss functions and metrics that quantify model performance. Super-resolution (SR) focuses on pixel fidelity and perceptual similarity, whereas segmentation emphasizes spatial overlap and classification accuracy.

2.4.1 Super-Resolution Objectives

Loss Functions. A loss function $L(\hat{Y}, Y)$ measures the discrepancy between the predicted image $\hat{Y}$ and the ground truth Y. Common choices include:

- **L1 Loss (Mean Absolute Error):**

$$L_{L1} = \frac{1}{N}\sum_{i=1}^{N}|\hat{Y}_i - Y_i|$$

  Encourages sharp reconstructions and is less sensitive to outliers than L2.

- **L2 Loss (Mean Squared Error):**

$$L_{L2} = \frac{1}{N}\sum_{i=1}^{N}(\hat{Y}_i - Y_i)^2$$

  Penalizes large deviations more strongly, leading to smoother but sometimes overly averaged outputs.

- **Charbonnier Loss (Smooth L1):**

$$L_{Charbonnier} = \frac{1}{N}\sum_{i=1}^{N}\sqrt{(\hat{Y}_i - Y_i)^2 + \epsilon^2}$$

  A differentiable L1 variant combining robustness to outliers with stable gradients near convergence [1].

Evaluation Metrics. Performance is quantified by reconstruction fidelity and perceptual similarity:

- **Peak Signal-to-Noise Ratio (PSNR):**

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{MAX_I^2}{\text{MSE}}\right)$$

  Higher PSNR indicates closer pixel-level correspondence.

- **Structural Similarity Index (SSIM):** Evaluates luminance, contrast, and structural similarity between reference and reconstructed images; ranges from -1 to 1 [14].

- **Multi-Scale SSIM (MS-SSIM):** Extends SSIM across multiple resolutions for perceptual robustness [15].

2.4.2 Segmentation Objectives

Loss Functions. Segmentation losses encourage accurate pixel classification and region overlap:

- **Binary Cross-Entropy (BCE):**

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

  Measures per-pixel classification error.

- **Dice Loss:**

$$L_{Dice} = 1 - \frac{2\sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i + \sum_i \hat{y}_i + \epsilon}$$

  Maximizes overlap between prediction and ground truth; robust to class imbalance [2].

Evaluation Metrics. Quantitative accuracy is measured using region overlap metrics:

- **Dice Coefficient:**

$$\text{Dice} = \frac{2 \times |P \cap G|}{|P| + |G|}$$

  Measures similarity between predicted and true masks [2].

- **Intersection-over-Union (IoU):**

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|}$$

  Standard in segmentation benchmarks such as PASCAL VOC [4].

2.5 Summary

This chapter presented the theoretical background required to understand the adaptive-depth U-Net methodology. It covered the fundamental principles of super-resolution and segmentation, explained the core deep learning mechanisms behind the U-Net, and described the loss functions and evaluation metrics used to assess model performance.

Chapter 3

Related Work

A crucial step in conducting thesis research is to review the existing literature and prior work on U-Net-based architectures for image super-resolution and medical image segmentation, as well as multi-scale and scale-aware deep learning methods relevant to the proposed adaptive-depth U-Net. This chapter will demonstrate awareness of the state of the art in the problem domain and position the proposed method as a novel contribution with respect to existing approaches.

3.1 Deep SISR

The earlier research work on SISR focused on very deep residual or iterative back-projection architectures, rather than U-Net-style encoder-decoders. For instance, Enhanced Deep Residual Networks for Single Image Super-Resolution(EDSR) [8], which introduced a highly optimized deep SR network that significantly surpassed previous state-of-the-art models. The key changes involved optimizing the conventional residual network (ResNet) architecture, primarily by removing the unnecessary Batch Normalization (BN) layers. By eliminating BN layers, the EDSR model could be substantially expanded in size (e.g., increased depth or feature channels), leading to improved quality results while maintaining manageable memory usage. In the context of U-Net–style encoder-decoder architectures, this motivates exploring how depth and capacity can be adjusted more systematically, instead of relying on a single, fixed configuration across all tasks and scales.
The Deep Back-Projection Network (DBPN) by Haris et al. [5] introduced a fundamental architectural shift in SISR by moving away from purely feed-forward modeling to incorporate an interactive error correction mechanism. So instead of upscaling once, it goes back and forth between the LR and HR multiple times and uses the reconstruction error itself to improve the result

This addresses a major limitation in previous Deep SISR architectures, as these fail to fully account for the mutual dependencies between the low-resolution input and high-resolution output.

From the perspective of this thesis, DBPN is interesting because it treats super-resolution as a multi-stage refinement process in which each stage contributes additional capacity to correct errors between the LR and HR. However, DBPN still uses a fixed number of projection stages that does not depend on the scale factor, and its architecture is not organized as a symmetric U-Net with skip connections at multiple resolutions.

3.2 U-Net Architectures for Medical Image Segmentation

Medical image segmentation, and skin lesion segmentation in particular, faces challenges such as low contrast between the lesion and the background, fuzzy boundaries, and large variation in lesion size and shape. The classical U-Net [12] addresses these issues with a symmetric encoder–decoder and skip connections that combine high-level context with fine spatial detail. Building on this baseline, Phan et al. propose “Skin Lesion Segmentation by U-Net with Adaptive Skip Connection and Structural Awareness [11], which introduces two key architectural changes:

Adaptive Skip Connections and a Structural Awareness Module. By integrating Selective Kernel blocks into the skip connections, the network can flexibly tune how much local versus global context it uses, depending on the lesion's size and appearance. While auxiliary distance-map and contour predictions encourage the model to be more sensitive to lesion boundaries and global shape.

This design can be seen as making the skip pathways of the U-Net scale-aware, improving performance on lesions of varying size, but the Overall encoder–decoder depth remains fixed. In contrast, the adaptive-depth U-Net explored in this thesis keeps the basic U-Net structure but treats the number of encoder–decoder levels as a scale-dependent design variable, focusing on how architectural depth itself should change with the lesion or image scale rather than only modifying the skip connections.

3.3 U-Net Architectures for Super-Resolution

Single-image super-resolution has been studied extensively using encoder–decoder (U-net-based) architectures for SR is effective because it combines semantic context with pixel-level detail reconstruction. Early deep SR research, such as [10], focused on parameter reduction, architecturally improved U-net, and the edge-sharpening MixGE loss function. The conclusion reached by Zhengyang Lu and Ying Chen is that it significantly improved performance compared to existing works, particularly by achieving superior reconstruction accuracy and efficiency.

3.3.1 Single-scale Super-Resolution Architectures

Single-scale Super-Resolution Architectures are neural networks specifically designed to reconstruct a HR image from LR input at a single predetermined scale factor. Researched Architectures like SRCNN, EDSR, ESPCN are single-scale architectures as they have a constant scale factor throughout the model.

Dong et al. propose "Image Super-Resolution Using Deep Convolution Networks " which was early research done in apply Deep CCN to Super-Resolution(SRCNN) and SRCNN layed the foundation for deep learning approach to image super-resolution[3]. The orgnial SRCNN architercutre was relatively shallow. This shallow depth subsequently limited its ability to model complex patterns and restore fine textures and high-frequency details effectively. When attempting to make deeper models, the authors observed challenges with superior performance, sometimes concluding that deeper networks did not yield better results. This observation was later successfully challenged by subsequent deep SR models, such as Very Deep Super-Resolution (VDSR)[7].

Shi et al. propose " Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network" which is an Efficient Sub-Pixel Convolution Neural Network (ESPCN), a resource efficient solution for single-image super-resolution designed for real-time application. ESPCN's efficiency stems from its use of sub-pixel convolution layer which is employed at the very end of the network.

3.3.2 Multi-Scale Super-Resolution Architectures

Accurate Image Super-Resolution Using Very Deep CNN (VDSR), which is a very deep CNN paper that uses a 20-layer convolutional network with residual learning, trained to predict the difference between a bicubic-upsampled low-resolution input and the ground-truth high-resolution image. In [7] Kim et al, show that a single network can be trained jointly on multiple scale factors (e.g. $\times 2$, $\times 3$, $\times 4$), achieving competitive performance across scales without maintaining separate models for each one used a joint multi-scale model. However, there is a massive computational and memory cost associated with training such a model as all convolutions operate

on large spatial dimensions, so FLOPs and memory usage are higher than methods that operate in LR space and only upscale at the end.

3.4 Summary and Research Gap

Chapter 4

Methodology

4.1 Hardware specification

- GPU: 1 × NVIDIA GeForce RTX 2080 Ti (11 GB VRAM)
- CPU: 4 Cores (allocated via SLURM)
- Memory: 31 GB RAM
- Partition: csedu (Radboud University Science Cluster)

4.2 Methodology Super Resolution

The methodology for this project is organized into four main components: the **Dataset and Preprocessing**, the **Adaptive-Depth U-Net Architecture**, the **Experimental Design**, and the **Training and Evaluation Protocol**.

The core objective is to investigate the optimal balance between the super-resolution downscale factor (s) and the architectural depth of the network.

4.2.1 Dataset and Preprocessing

The **DIV2K** dataset, a standard benchmark for single-image super-resolution (SR), was used for training and evaluation. Specifically, the **Train Data Track 1 – Bicubic Downscaling** $\times 4$ subset was utilized, consisting of 800 high-resolution (HR) and low-resolution paired (LR) images for training, and 100 pairs for validation. HR images are approximately 2040×1080 pixels before cropping, and the LR inputs are 510×270 pixels (downsampled by a factor of 4).

Low-Resolution (LR) Data Generation

Two primary strategies were considered for generating the low-resolution (LR) training inputs:

Synthetic (On-the-fly) Generation This approach loads only the DIV2K High-Resolution (HR) frames. The `tf.data` pipeline then crops random 256×256 patches and immediately applies a bicubic downsample/upsample operation to each patch. This strategy keeps the workflow simple, as no extra assets are required. To maintain consistency, I set the LR degradation to $0.5x$ by default for comparability across scales. The LR inputs are therefore generated dynamically while remaining perfectly pixel-aligned with the HR patches.

Precomputed (Official) Tiles This alternative involves using the "official" bicubic-downsampled LR sets provided by DIV2K (e.g., `X2`, `X3`, `X4`). This would require downloading the corresponding directories and pairing every HR crop with its stored LR counterpart. While the benefit is matching the benchmark's exact degradation pipeline, this method is limited to the provided scales (e.g., 0.5 for $\times 2$, ≈ 0.33 for $\times 3$, 0.25 for $\times 4$) and incurs extra I/O and data management overhead.

For this project, the **Synthetic (On-the-fly) Generation** method was adopted. All models are trained using LR patches dynamically generated from the HR images, eliminating the need for additional LR datasets.

4.2.2 U-Net Architecture

Vanillin UNet model This is the classic 4-level encoder-decoder with symmetric skip connections. Each encoder block consists of a Conv2D → BatchNorm → Relu and then halves the spatial resolution with MaxPool2D. Each decoder block consists of UpSampling2D (with bilinear interpolation) → Conv2D → concatenation of the corresponding skip connections. The head is a single 1×1 convolution (with sigmoid activation) that directly predicts the HR RGB patch.

Modified U-Net with a Residual Head The selected architecture is a modified **U-Net** [12], chosen after reviewing prior work. It provides a strong baseline for testing both super-resolution quality and, more importantly, the effectiveness of the proposed adaptive-depth architecture. To enhance reconstruction stability, a **Residual Reconstruction Head** inspired by residual learning [6] is applied at the output. The final reconstruction $\hat{Y}$ is computed as:

$$\hat{Y} = \text{ClippedResidualSum}(X_{\text{LR}}, \text{Residual}(X_{\text{LR}}))$$

where X_{LR} denotes the low-resolution input and Residual$(\cdot)$ represents the learned correction. The `ClippedResidualSum` layer ensures that all pixel intensities remain within valid image bounds after residual addition.

To support arbitrary scaling during both upsampling and downsampling, two custom TensorFlow layers were implemented: `ResizeByScale` and `ResizeToMatch`. Unlike fixed-stride pooling or transposed convolutions, these layers perform continuous spatial scaling using differentiable bilinear interpolation, allowing the model to handle non-integer scale factors while maintaining smooth gradient flow.

Adaptive-Depth Principle The main idea behind the adaptive-depth design is that the network depth should change with the input scale factor s. When the input is heavily downscaled, a deep encoder–decoder quickly reduces feature maps to very small sizes, leading to loss of spatial detail. For such cases, a shallower network is more efficient and stable. At higher scales, more spatial information is available, allowing deeper models to extract richer features. However, this increased spatial resolution also raises computational and memory requirements, often resulting in longer training times or the need for smaller batch sizes. Based on this intuition, the U-Net depth is adjusted as follows:

- **Small scales ($s \leq 0.4$):** Use fewer encoder–decoder levels to preserve detail and reduce computation.
- **Large scales ($s \geq 0.5$):** Use more levels to capture richer features, at the cost of higher training time and memory use.

4.2.3 Experimental Design

Two primary experiments were designed to isolate and analyze the effects of scale factor and network depth. All models were trained using the **Charbonnier Loss** (a robust form of L1 loss) for stable optimization.

Experiment 1: Fixed Depth, Varying Scale

This experiment establishes a baseline performance across a wide range of scale factors (s) using a uniform network capacity.

- **Network Depth:** Fixed at **3 encoder-decoder levels** for all runs.
- **Scale Factors (s):** Varied across a broad spectrum: $\{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$.

The results from this experiment reveal the inherent performance ceiling of a fixed architecture when faced with increasingly higher-fidelity inputs.

Experiment 2: Adaptive Depth per Scale

This is the core test of the proposed adaptive architecture. The network depth is tuned for each scale factor based on the principle described in Section 4.2.2, aiming to optimize feature map size stability and capacity.

Table 4.1: Adaptive-Depth Configuration per Scale Factor

Scale (s)	**Depth (Levels)**	**Rationale**
0.20	1	Maintain feature maps ≥ 50 px
0.30	2	Adds capacity; avoids map collapse
0.40	3	Balanced configuration
0.50	3	Baseline stability
0.60	4	Increased capacity for higher detail
0.70	5	Significant capacity increase
0.80	5	High-detail retention

The performance of these adaptive models is benchmarked against the fixed-depth models (Experiment 1) and a **Bicubic Baseline** (i.e., the LR image upsampled by bicubic interpolation to HR size).

4.3 Methodology: Segmentation

4.3.1 Dataset and Preprocessing

Dataset ISIC-2017 lesion segmentation dataset is a common dataset used in research into segmentation and U-Net design. The selection of this dataset was done as it was easily comparable to the reearch in the related work section. The dataset consists of 3 splits train (2000 images), test (600 images), and valid (150 images).

Preprocessing All images and masks undergo the following standardization before entering the network

Input Images:

- Resizing: Images are resized to a fixed input size of 256 × 256 pixels using Area interpolation (which is preferred for downsampling to avoid aliasing artifacts).
- Normalization: Pixel values are normalized to the range [0, 1] (converted to float32).
- Channels: Loaded as 3-channel RGB images.

Segmentation Masks

- Resizing: Resized to 256×256 pixels using Nearest Neighbor interpolation. This is crucial to preserve the discrete nature of the class labels and avoid introducing intermediate values at boundaries.
- Binarization: The masks are thresholded at 0.5 to ensure strict binary values (0.0, 1.0), representing background and lesion respectively.

Data Augmentation To prevent overfitting, the following augmentations are applied dynamically during training (and disabled during validation/testing):

- Random Rotation: The image and mask are rotated by 0, 90, 180, or 270 degrees ($k \in 0, 1, 2, 3$).
- Random Flips: Horizontal Flip applied with 50% probability and Vertical Flip applied with 50% probability.
- Random Scale qne Crop:The image is scaled up by a random factor between 1.0x and 1.15x. It is then randomly cropped back to the original size (256×256).
- Note: Bilinear interpolation is used for scaling images, while Nearest Neighbor is used for masks.

Pipeline Optimization The data is loaded using the tf.data API with the following performance optimizations:

- Shuffling: The training dataset is fully shuffled at each epoch. Parallel Mapping: Preprocessing and augmentation are parallelized ($num\_parallel\_calls = tf.data.AUTOTUNE$)
- Prefetching: Data is prefetched to the GPU memory to prevent I/O blocking.

4.3.2 U-Net Architecture

Vanilla U-Net

1. **Core Structure**: The model follows the classic Encoder-Decoder (U-Net) architecture with skip connections.
   - Input: RGB Images of shape $(256, 256, 3)$.
   - Output: Binary segmentation mask of shape $(256, 256, 1)$.
2. **Building Blocks**: The architecture is built using a standard Convolutional Block (`conv_block`) repeated throughout the network.
   - **Double Convolution**: Each block consists of two consecutive sets of:
     - `Conv2D`: 3×3 kernel, `padding="same"` (preserves spatial dimensions).
     - `LayerNormalization`: Applied over the channels (`axis=-1`).
     - `ReLU Activation`: Non-linear activation function.
3. **Encoder (Contracting Path)**: The encoder progressively reduces spatial resolution while increasing feature depth.
   - **Depth**: Configurable, default is 4 levels.
   - **Downsampling**: Uses `MaxPooling2D` with a 2×2 pool size.

- **Feature Channels**: Starts with `base_channels` (default 32) and doubles at each level ($32 \to 64 \to 128 \to 256$).
- **Skip Connections**: The output of each encoder block (before pooling) is saved to be concatenated with the corresponding decoder level.

4. **Bottleneck**: A single `conv_block` at the bottom of the "U".
   - Contains the highest number of filters (e.g., 512 if depth is 4).
5. **Decoder (Expansive Path)**: The decoder recovers spatial resolution to generate the segmentation mask.
   - **Upsampling**: Uses `Conv2DTranspose` (Deconvolution) with a 2×2 kernel and stride 2.
   - **Concatenation**: The upsampled feature map is concatenated with the corresponding skip connection from the encoder.
   - **Processing**: Followed by a `conv_block` to process the combined features.
   - **Feature Channels**: Halves at each level as it goes up.
6. **Output Layer**
   - **Convolution**: A final `Conv2D` with a 1×1 kernel.
   - **Activation**: Sigmoid activation to squash values between $[0, 1]$ for binary classification (pixel-wise probability).

Adaptive-Depth U-Net Architecture

1. **Normalization Strategy**
   - **Adaptive U-Net**: Uses `BatchNormalization`. This normalizes across the entire batch, which is generally standard for CNNs but sensitive to batch size.
   - **Vanilla**: Used `LayerNormalization` (normalizes per sample).
2. **Upsampling Method**
   - **Adaptive U-Net**: Uses Bilinear Upsampling (`UpSampling2D`). This is a fixed, non-learnable interpolation operation. It is often smoother and avoids the "checkerboard artifacts" sometimes caused by learnable deconvolutions.
   - **Vanilla**: Used `Conv2DTranspose` (learnable deconvolution).
3. **Regularization (Dropout)**
   - **Adaptive U-Net**: Includes `SpatialDropout2D`. It drops entire feature maps (channels) rather than individual pixels, which is more effective for convolutional feature maps where adjacent pixels are highly correlated.
     - Applied after the bottleneck.
     - Applied in the decoder blocks after concatenation.
   - **Vanilla**: Had no dropout.
4. **Depth Flexibility**
   - **Adaptive U-Net**: While both scripts accept a `depth` argument, this script explicitly names the model `adaptive_unet_depth{N}`, highlighting that the architecture is designed to be dynamically scaled (e.g., changing depth to 3, 4, or 5) as a primary hyperparameter for the experiments.

4.3.3 Running Optuna to Obtain the Best Hyperparameter

The segmentation model's hyperparameter were optimized using the **Optuna** framework, which performed a Bayesian search over a predefined parameter space. The best-performing configuration was obtained after multiple trials and provides us with a good base for training your Segmentation model, yielding the following parameters:

- **Learning rate:** 3.0592×10^{-5}
- **Base channels:** 32
- **Depth:** 5
- **Batch size:** 8
- **Data augmentation:** True

4.3.4 Experimental Design

Similar approach to SR was implemented with the loss function being Dice and IoU.

Experiment 1: Fixed Depth, Varying Scale

This experiment establishes a baseline performance across a wide range of scale factors (s) using a uniform network capacity with total parameters being 119.79MB with 119.75MB being trainable and 46KB being non-trainable.

- **Network Depth:** Fixed at **4 encoder-decoder levels** for all runs.
- **Scale Factors (s):** Varied across a broad spectrum: $\{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$.

Experiment 2: Adaptive Depth per Scale

This is the core test of the proposed adaptive architecture. The network depth is tuned for each scale factor based on the principle described in Section 4.2.2, aiming to optimize feature map size stability and capacity.

Table 4.2: Adaptive-Depth Configuration per Scale Factor

Scale (s)	**Depth (Levels)**	**Rationale**
0.20	1	
0.30	2	
0.40	3	
0.50	3	
0.60	4	
0.70	5	
0.80	5	

Chapter 5

Results and Analysis

This chapter presents the quantitative and qualitative results of the experiments outlined in the methodology. We evaluate the performance of both the **Fixed-Depth** and **Adaptive-Depth U-Net** models against a standard Bicubic interpolation baseline. All metrics are reported on the 256×256 cropped DIV2K validation set, using a 4-pixel border shave on the luminance (Y) channel.

5.1 Experiment 1: Fixed-Depth Model Performance

The first experiment evaluates the performance of a U-Net with a **fixed depth of 3 levels** across all scale factors. The objective is to establish a performance baseline for a non-adaptive architecture. The results are compared against the bicubic baseline in Table 5.1.

Table 5.1: Quantitative Results for Fixed-Depth (3 Levels) U-Net vs. Bicubic Baseline. The Δ column shows the improvement of our model over bicubic interpolation.

Scale	**Fixed**	**PSNR (dB)**			**SSIM**			**MS-SSIM**
(s)	**Depth**	**Bicubic**	**Model**	**Δ**	**Bicubic**	**Model**	**Δ**	**(Model)**
0.20	3							
0.30	3							
0.40	3							
0.50	3							
0.60	3							
0.70	3							
0.90	3							

Analysis of Fixed-Depth Results (TODO: Your analysis here. Discuss the trends from Table 5.1. Note the general improvement over bicubic and any potential performance bottlenecks at extreme scales.)

5.2 Experiment 2: Adaptive-Depth Model Performance

The second experiment tests the core hypothesis: that adapting the network depth based on the scale factor improves performance. Deeper networks are used for higher-detail scales, while shallower networks are used for low-detail scales. The results are shown in Table 5.2.

Analysis of Adaptive-Depth Results (TODO: Your analysis here. Discuss the results from Table 5.2, focusing on the Δ vs. Bicubic improvements. Set the stage for the direct comparison in the next section.)

Table 5.2: Quantitative Results for **Adaptive-Depth U-Net** vs. Bicubic Baseline. The depth is now a function of the scale s.

Scale	**Adaptive**	**PSNR (dB)**			**SSIM**			**MS-SSIM**
(s)	**Depth**	**Bicubic**	**Model**	**Δ**	**Bicubic**	**Model**	**Δ**	**(Model)**
0.20	1							
0.30	2							
0.40	3							
0.50	3							
0.60	4							
0.70	5							
0.80	6							
0.90	7							

5.3 Comparative Analysis: Fixed vs. Adaptive Depth

This section provides the direct comparison to answer the primary research question. Table 5.3 consolidates the model performance from both experiments to highlight the net benefit (or cost) of the adaptive-depth strategy.

Table 5.3: Direct Performance Comparison: Fixed-Depth (D=3) vs. Adaptive-Depth Model. The Δ columns represent the performance gain of the adaptive model over the fixed model.

Scale	**PSNR (dB)**			**SSIM**		
(s)	**Fixed (D=3)**	**Adaptive**	**Δ (Adaptive - Fixed)**	**Fixed (D=3)**	**Adaptive**	**Δ (Adaptiv**
0.20						
0.30						
0.40						
0.50						
0.60						
0.70						
0.80						
0.90						

Discussion of Comparative Results (TODO: This is your most important analysis. Use the Δ columns in Table 5.3 to prove or disprove your hypothesis.

- At low scales ($s \leq 0.4$): Does the adaptive model win?
- At high scales ($s \geq 0.6$): Does the adaptive model win?
- At crossover scales ($s = 0.4, 0.5$): Is the performance similar, as expected?

)

5.4 Analysis of Training Dynamics

Beyond the final checkpoint metrics, analyzing the training and validation loss curves provides insight into model stability, convergence speed, and overfitting.

5.4.1 Experiment 1: Fixed-Depth Training

Figure 5.1 plots the validation PSNR (or loss) over training epochs for the fixed-depth (D=3) model at different scales.

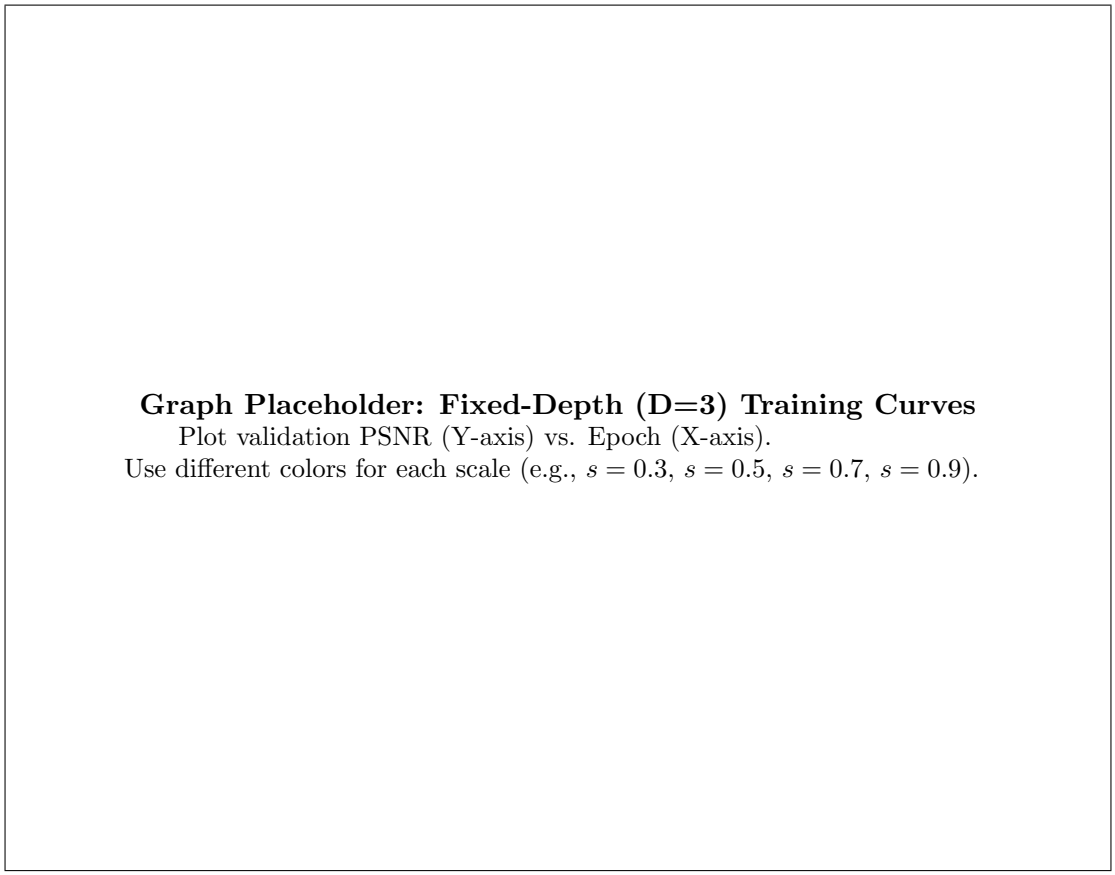

Figure 5.1: Validation PSNR curves for the **Fixed-Depth (D=3)** model across various scales. (You may also want a second plot showing the train/validation loss gap for a specific scale).

Analysis of Fixed-Depth Training (TODO: Your analysis here.

- Do higher scales (e.g., 0.9) converge faster or slower?
- Do they plateau at a higher or lower PSNR?
- Do the low scales (e.g., 0.2, 0.3) show signs of overfitting (a large gap between training and validation loss) because the model is too complex for the simple task?)

)

5.4.2 Experiment 2: Adaptive-Depth Training

Figure 5.2 shows the same plot, but for the adaptive-depth models. This compares, for example, the convergence of the $s = 0.3$ (D=2) model against the $s = 0.7$ (D=5) model.

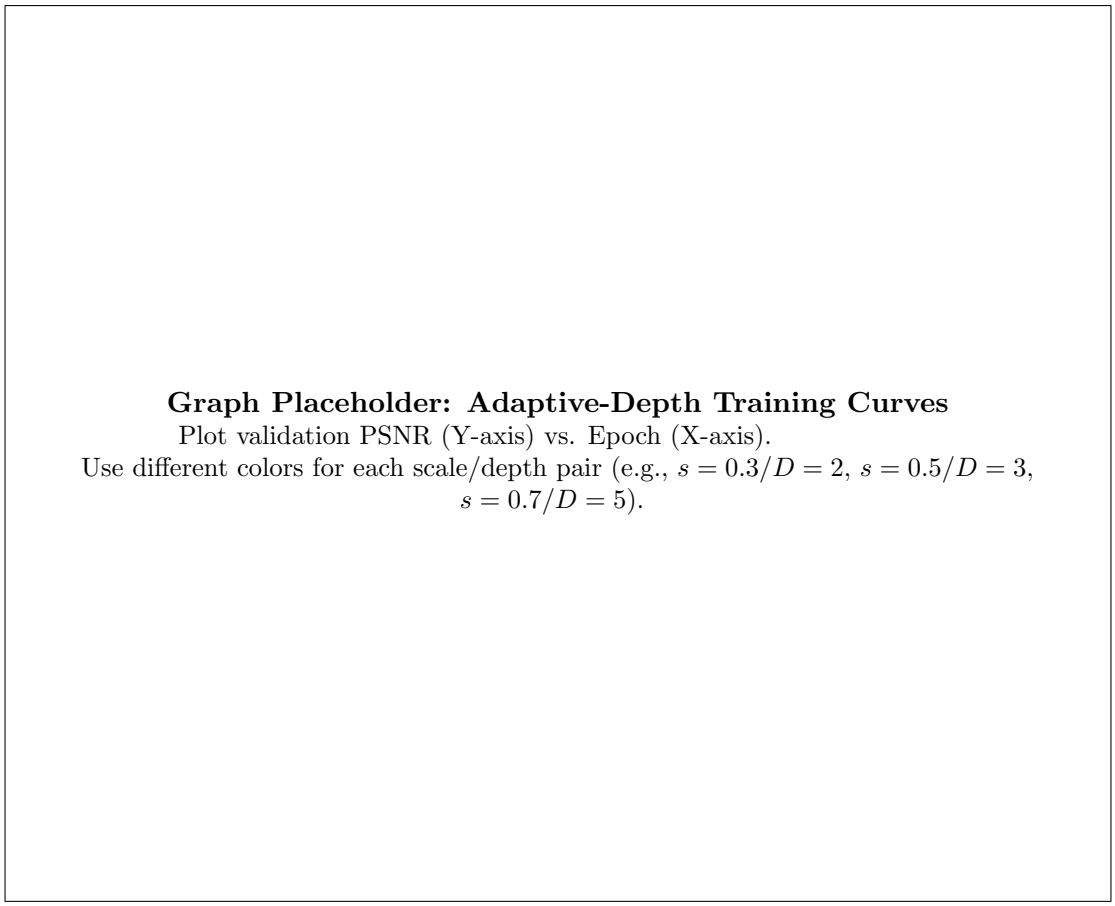

Figure 5.2: Validation PSNR curves for the **Adaptive-Depth** models across various scales.

Analysis of Adaptive-Depth Training (TODO: Your analysis here.

- How do these curves compare to the fixed-depth ones in Figure 5.1?
- Does the $s = 0.3$ (D=2) model show a smaller validation gap than the $s = 0.3$ (D=3) model from the first experiment (suggesting less overfitting)?
- Does the $s = 0.7$ (D=5) model converge to a higher validation PSNR than the $s = 0.7$ (D=3) model?
- Are the training curves generally more stable?)

)

5.5 Qualitative Results (Visual Comparison)

Quantitative metrics provide an objective measure of error, but qualitative (visual) inspection is necessary to assess perceptual quality, such as texture reconstruction and artifact suppression. Figures 5.3 and 5.4 show visual comparisons on selected patches from the validation set.

Analysis of Qualitative Results (TODO: Discuss the visual evidence from your figures. Point to specific areas (e.g., "in Figure 5.4 (d), the textures..."). Explain *how* the visual results support the quantitative findings in your tables.)

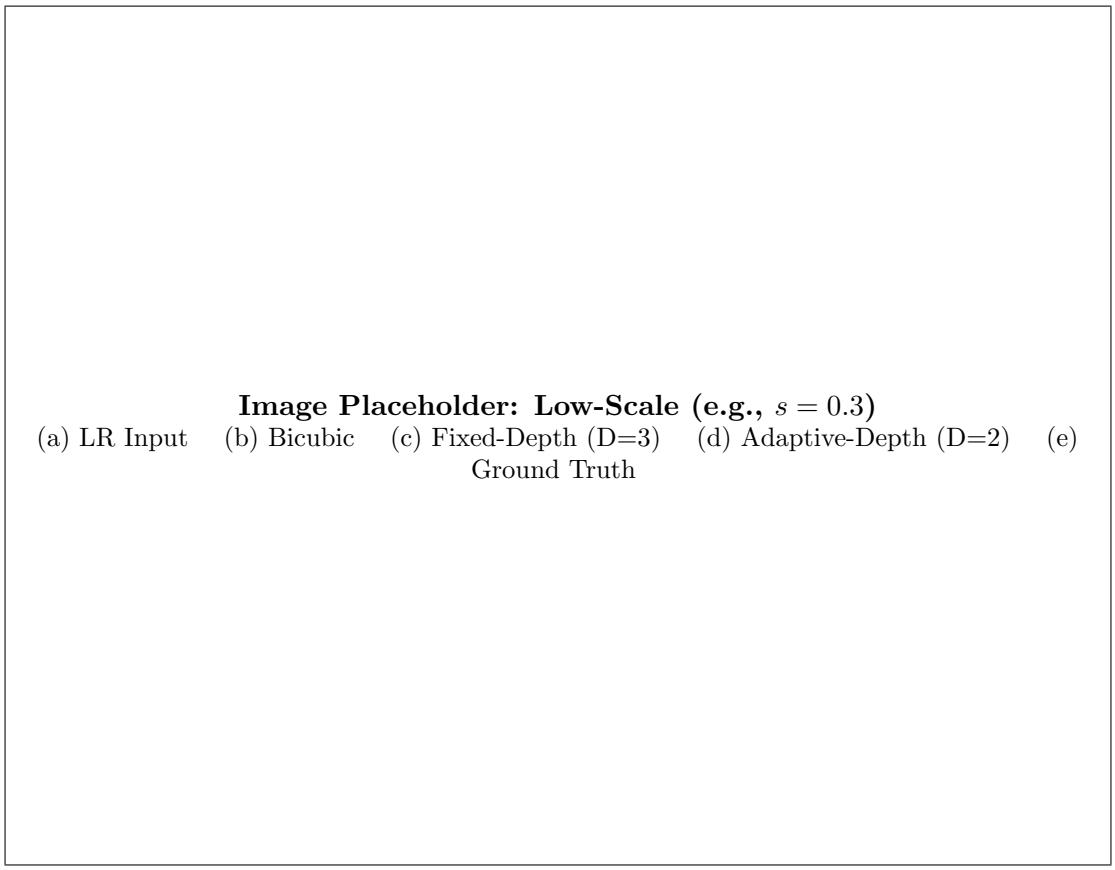

Figure 5.3: Visual comparison on a low-detail scale ($s = 0.3$).

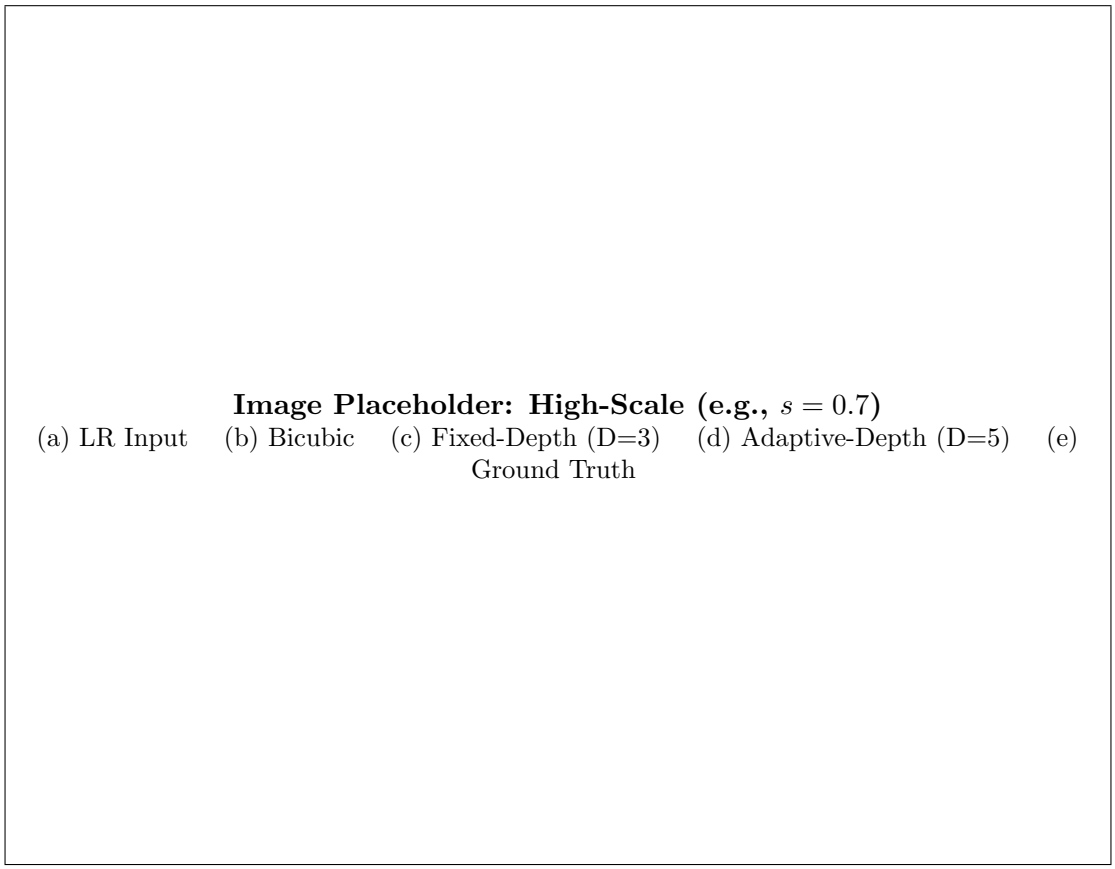

Figure 5.4: Visual comparison on a high-detail scale ($s = 0.7$).

Chapter 6

Discussion and Limitations

Chapter 7

Conclusions

In this chapter you present all conclusions that can be drawn from the preceding chapters. It should not introduce new experiments, theories, investigations, etc.: these should have been written down earlier in the thesis. Therefore, conclusions can be brief and to the point.

Bibliography

[1] Pierre Charbonnier, Laure Blanc-Feraud, Gilles Aubert, and Michel Barlaud. Two-dimensional disparity estimation using a hierarchical approach. *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2:168–172, 1994.

[2] Lee R. Dice. Measures of the amount of ecologic association between species. *Ecology*, 26(3):297–302, 1945.

[3] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Image super-resolution using deep convolutional networks. 2015.

[4] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[5] Muhammad Haris, Greg Shakhnarovich, and Norimichi Ukita. Deep back-projection networks for super-resolution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[7] Jiwon Kim, Jung Kwon Lee, and Kyoung Mu Lee. Accurate image super-resolution using very deep convolutional networks. pages 1646–1654, 2016.

[8] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, and Kyoung Mu Lee. Enhanced deep residual networks for single image super-resolution.

[9] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440, 2015.

[10] Zhengyang Lu and Ying Chen. Single image super resolution based on a modified u-net with mixed gradient loss. *arXiv preprint arXiv:1911.09428*, 2019.

[11] Tran-Dac-Thinh Phan, Soo-Hyung Kim, Hyung-Jeong Yang, and Guee-Sang Lee. Skin lesion segmentation by u-net with adaptive skip connection and structural awareness. *Sensors*, 23(2):889, 2023.

[12] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Lecture Notes in Computer Science. Springer, 2015.

[13] Yifan Wang, Tian Chen, and Shuang Zhang. Interpolation-based pooling for efficient feature rescaling in convolutional networks. *IEEE Access*, 8:123456–123467, 2020.

[14] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[15] Zhou Wang, Eero P. Simoncelli, and Alan C. Bovik. Multi-scale structural similarity for image quality assessment. *Proceedings of the 37th IEEE Asilomar Conference on Signals, Systems and Computers*, 2:1398–1402, 2003.

Chapter A

Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.

A.1 Custom Layers and Utility Functions